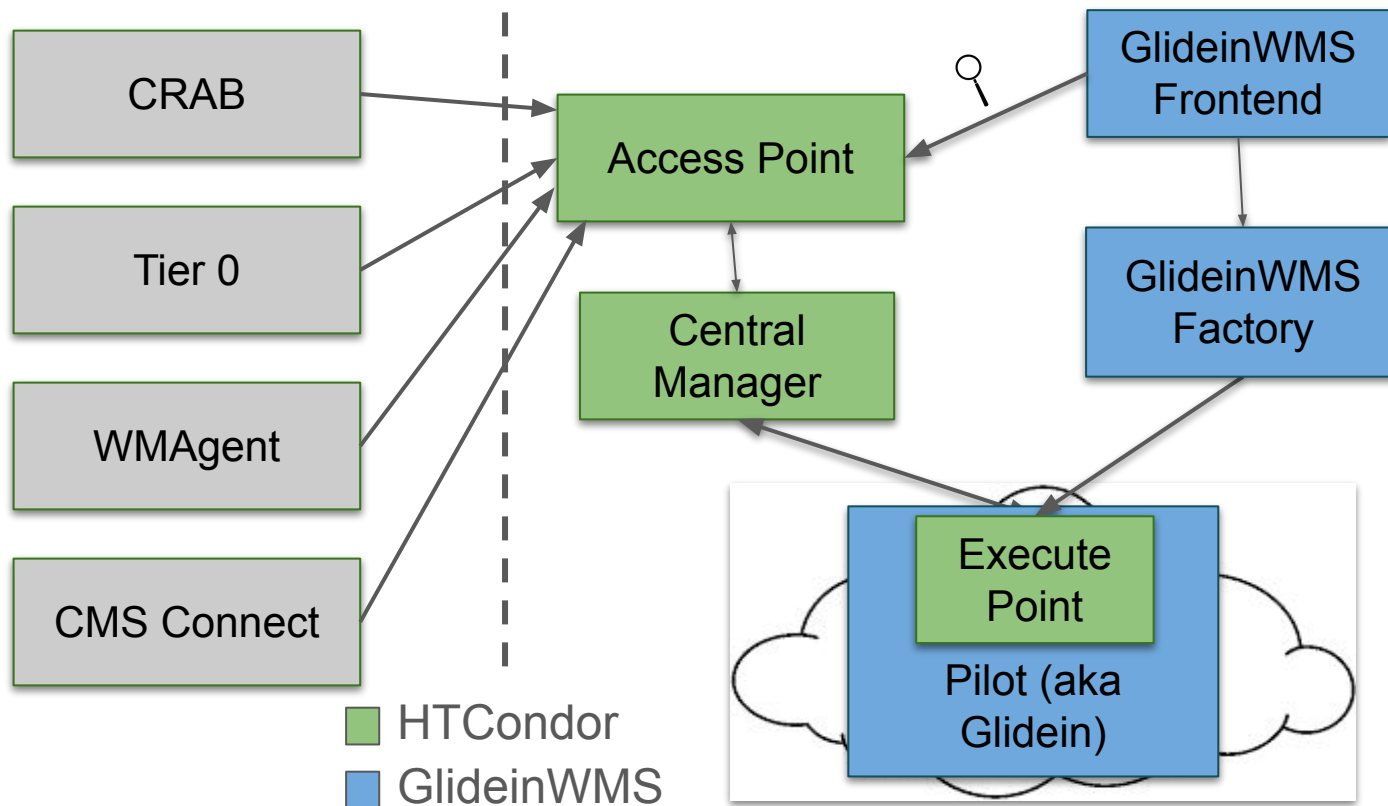


Update from Submission infrastructure

Marco Mascheroni, A. Pérez-Calero Yzquierdo, Ralf Florian
Von Cube, Hyun Woo Kim, Vaiva Zokaite

The CMS SI basic setup



Alma 9 Transition

Transition Strategy Overview

Goal: Seamless transition of CMS Submission Infrastructure (SI) to ALMA9 while maintaining operational stability.

- Transition needed for uninterrupted data-taking and processing.
- High availability to ensure no production downtime.

Transition Stages:

1. **Testing on ALMA9:** Deployed HTCondor on test machines, resolved issues.
2. **Gradual Transitioning:** Phased transition of pools (ITB, Global, CERN) with testing to validate readiness.

Central Managers & APs Transition

Central Managers Transition (Collectors, Negotiators):

- Smoothly switched between CERN and Fermilab collectors/negotiators to ensure redundancy.
- Updated configurations for primary and backup roles.

APs Transition:

- **Setup & Testing:** Provided ALMA9 access points (APs) to production, analysis, and Tier-0 groups.
- Coordinated adjustments in AP configurations for efficient use, ensuring memory and job limits were optimized.
- All schedds transitioned without major disruptions.

No Impact on CMS Operations:

- Transition left CMS resources (~500k CPU cores) stable and unaffected.
- Smooth migration of both central machines and APs.
- Modularity and high availability of HTCondor allowed seamless, phased replacement of services.

Transition to HTC v2 Python Bindings

CRAB as Alpha Tester for CMS Analysis Tasks

- Gained early access to `pre-release` code in March.

Positive Experience with Upgrades

- Prompted updates to code that hadn't been touched in over 6 years, improving clarity and removing deprecated elements.
- Cleaned up codebase; Python API improvements over the years streamlined adaptation.
- Updating to v2 bindings took just **1 day**—main change was in `schedd.spool()` semantics, now clearer.

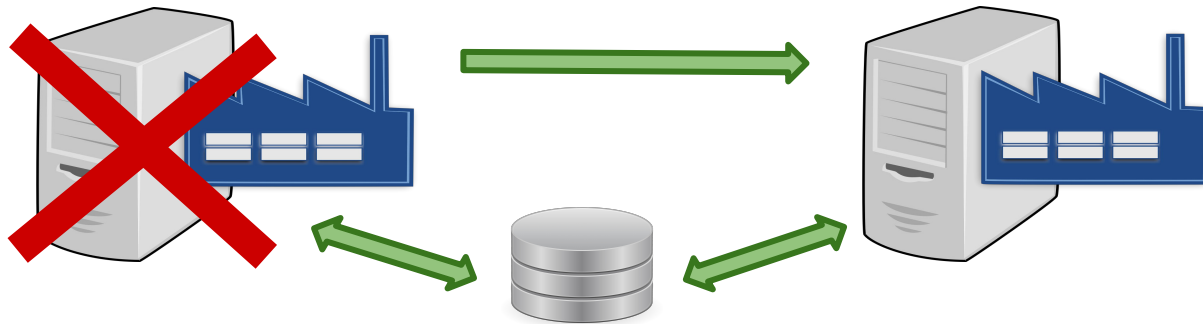
Collaboration & Support

- Supported the development process by helping identify minor issues.
- Appreciative of excellent support, especially from ToddM!

Factory to kubernetes

Motivation:

- Automating scaling, load balancing, and failover processes: **high availability and resilience**.
- Part of a larger effort within OSG to modernize the infrastructure.
 - OSG services such the GRACC accounting system, glidein pool frontends and central managers, hosted compute entrypoints (CEs), software repositories, and web pages have also undergone similar migrations.

**Challenges:**

- The Kubernetes (K8s) factory pod installs itself from scratch every restart
 - **Any state must be saved between restarts**. GlideinWMS and HTCondor both depend on state written to disk
 - The factory must run a reconfig command automatically on startup

Persistent state:

- K8s provides a Persistent Volume Claim (PVC) object that allows us to **retain state between pod restarts**
- We created **3 NVMe backed Ceph volumes** to save state for:
 - `/var/lib/condor`
 - `/var/lib/gwms-factory`
 - `/var/log/gwms-factory`
- These volumes can be increased in size as needed **without requiring a pod restart**

Improved automations

Updates to Reconfig Procedure

Old bare metal way

- Update file(s) in /etc/osg-gfactory/
- Manually run the following commands:
systemctl stop gwms-factory
 - gwms-factory reconfig
 - systemctl start gwms-factory



Updates to download of HTCondor Tarballs

Old bare metal way

- Manually download new tarballs from <https://research.cs.wisc.edu/htcondor/tarball/> into /var/lib/gwms-factory/condor/
- Run reconfig procedure described above

New K8s way

- Update files(s) in osg-gfactory Git repo
- Cron script fires off every 15 min to check for changes
- If changes are detected, script automatically stops, reconfigures, and restarts the factory

New K8s way

- Update new tarball config yaml file in Git repo
- Reconfig hook* runs the get_tarball script which automatically downloads any tarballs in config that the factory does not yet have

CPU utilization efficiency and overloading (CHEP24)

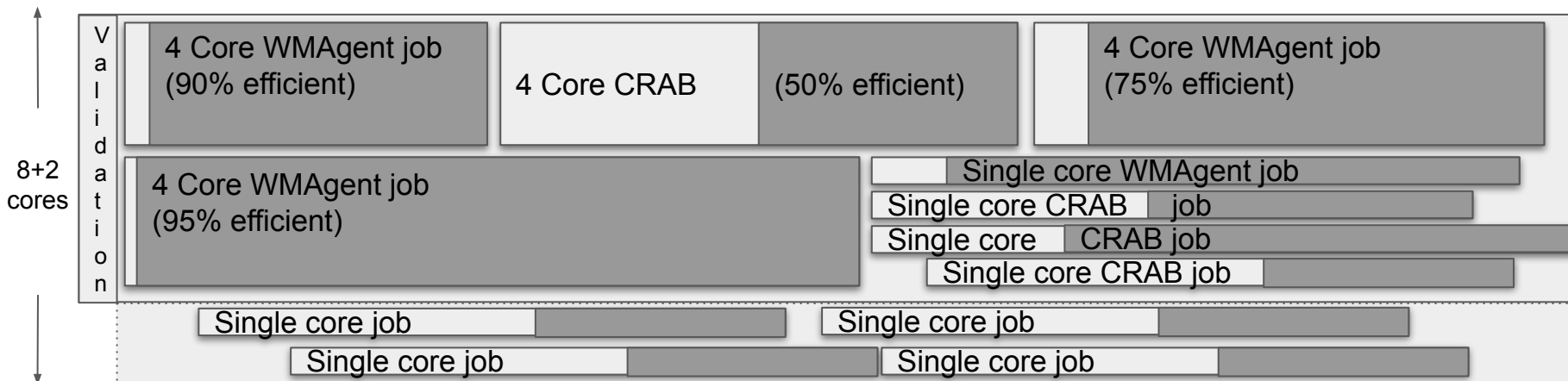
Strategy to recover unused CPU cycles: overloading pilots

Idea: Re-definition of the efficiency problem:

- **Improve CPU utilization efficiency by pushing more workload into the same pilot envelope**
 - Modify pilots so that **they accept more payload jobs into the same resources**
 - **Trivial to implement and test from CMS Submission Infrastructure side**

Principle: we want to recover unused CPU, **not gain opportunistic cycles!**

- **Moderate overloading:** add 25% extra CPU cores and memory to the nominal values of our standard 8-core pilot. Provides 2 extra cores, e.g. available to run additional CRAB or production payload



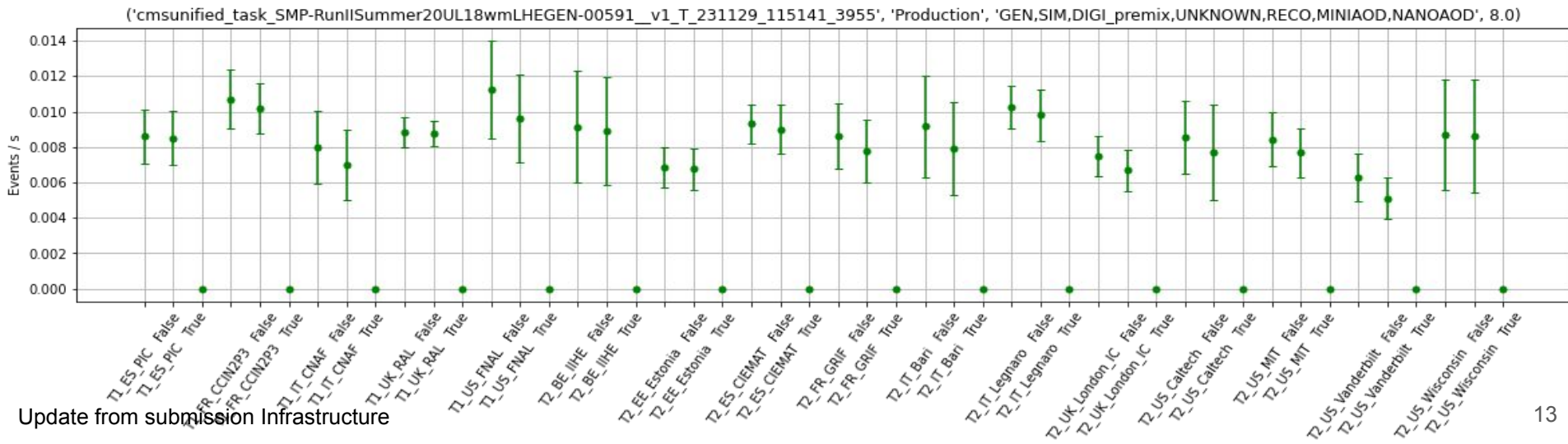
Efficiency Improvements

- From the pilot logs, total walltime and used CPU time can be extracted. We can thus calculate the CPU efficiency as measured by the resource providers and reported to EGI
- Efficiencies based on pilot logs executed over last 3 months can be used to compare overloaded vs non-overloaded pilots
- **Significant improvement** of CPU utilization efficiency when allowing overloaded pilots

Site Name	Normal Efficiency	Overloaded Efficiency	Efficiency increase
T1_FR_CCIN2P3	84.57%	95.78%	11.21%
T1_IT_CNAF	79.81%	84.62%	4.81%
T1_UK_RAL	72.41%	85.00%	12.60%
T2_BE_IHHE	77.21%	89.17%	11.96%
T2_DE_RWTH	55.68%	78.01%	22.33%
T2_EE_Estonia	73.81%	85.47%	11.66%
T2_ES_CIEMAT	72.65%	88.40%	15.75%
T2_IT_Bari	74.48%	78.87%	4.39%
T2_IT_Legnaro	75.77%	85.88%	10.11%
T2_IT_Rome	60.25%	73.38%	13.13%
T2_UK_London_IC	66.14%	72.94%	6.80%
T2_US_MIT	63.74%	70.61%	6.87%
T2_US_UCSD	52.91%	67.64%	14.73%
T2_US_Vanderbilt	74.95%	77.04%	2.09%
T2_US_Vanderbilt	56.05%	65.64%	9.58%
T2_US_Wisconsin	78.03%	89.66%	11.62%

Event Rates comparison

- Compared event rate results for all workflows for several months, **classifying jobs by execution site** and workflow type.
- First example, notice this full **StepChain simulation workflow** (~450k jobs in total)
 - Results: event processing rates present **high variability**, with **overloading effect** on throughput **smaller than dispersion** between jobs at the same site, and across sites



Overall impact on fully loaded pilots

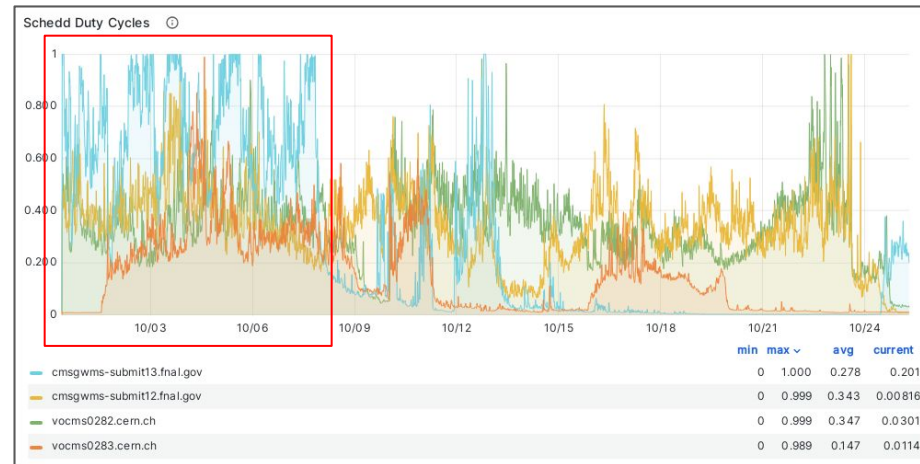
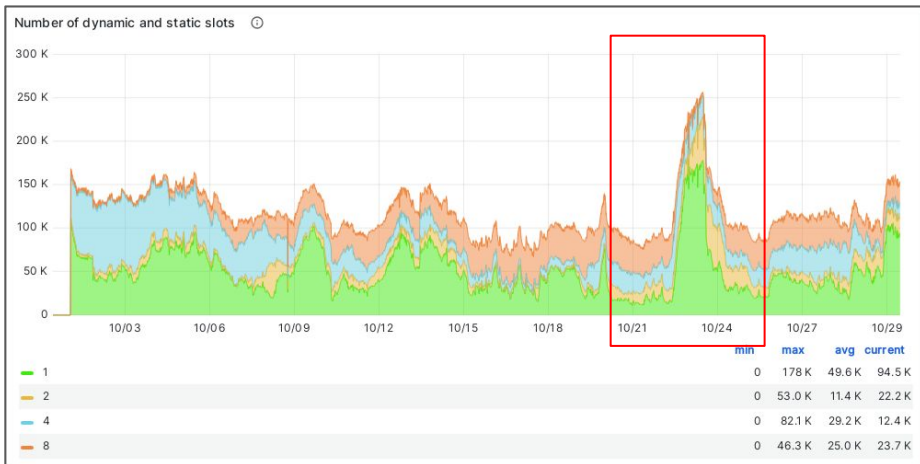
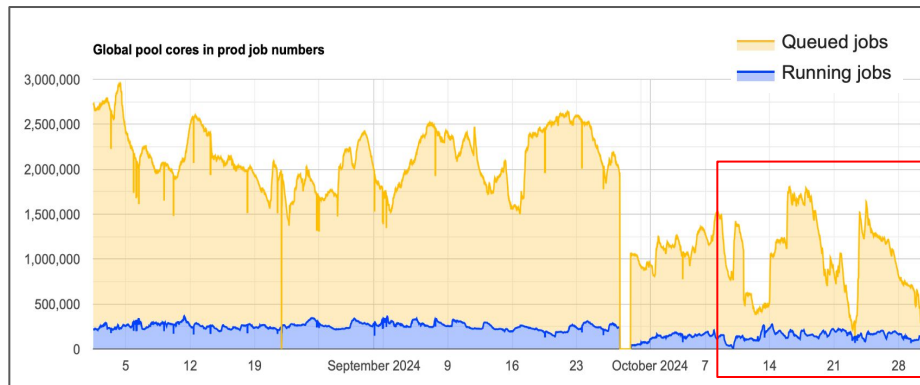
Measured total number of events produced by a single pilot running on 8 physical cores as either 8 or 10 processes.

Increased total event throughput. Slower event rate per process is over compensated by running more processes.

Site	Processor	On (Evt/s)	Off (Evt/s)	Evt/s increment %
T2_BE_IHHE	AMD EPYC 7452 32-Core Processor	3,14	2,82	11,35
T2_DE_RWTH	AMD EPYC 7543 32-Core Processor	2,89	2,49	16,06
T2_ES_CIEMAT	Intel(R) Xeon(R) Gold 5318Y CPU @ 2.10GHz	4,31	3,58	20,39
T2_IT_Bari	AMD EPYC 7413 24-Core Processor	2,96	2,81	5,34
T2_IT_Legnaro*	AMD EPYC 7282 16 7413 24-Core Processor	2,98	2,74	8,76
T2_UK_London_IC	Intel(R) Xeon(R) CPU E5-2620 v4 @ 2.10GHz	1,70	1,61	5,59
T2_US_UCSD	AMD EPYC 7662 64-Core Processor	2,42	1,63	48,47
T2_US_Vanderbilt	Intel(R) Xeon(R) CPU E5-2420 0 @ 1.90GHz	1,30	1,05	23,81

CPU usage efficiency: impact from WM and Ops

- Single core storms affecting the pools performance
- Lack of (enough) work in the schedd queues
- Saturated WMAgents



Future works and Conclusions

Projects for 2025

- Transition to tokens:
 - SciToken, ARC-CEs (outlier sites that still want to keep using proxies as long as supported per WLCG policy)
 - Deploy CredMon condor credential. Be prepared to handle WMAgent and CRAB tokens when proxies will be deprecated for data management.
- Efficiency and utilization of resources:
 - Complete pilot overloading deployment for increased CPU usage efficiency
 - Overhauling of removal expressions mechanism profiting from a new feature in HTCondor23
 - Improved monitoring: SI historical monitoring, new dashboard to view the pilot efficiency, migration of the gwmsmon service to Alma9 and monitoring service to retrieve pilot logs
 - Enabling local config for CMS pilots from sites for improved matchmaking, resource type monitoring and the integration of SONIC [DONE]
- Resource Allocation:
 - Pilots discriminating on CPU microarchitecture (exclude v1, DONE)
 - Evaluation of the Decision Engine as a potential replacement for GlideinWMS frontend
 - Enabling resource integration through Containerization (provide site admins with a container to launch on WNs)
 - Factory scale test for HL-LHC
- Migration of the infrastructure to HTCondor24

Conclusions

Successful Transition to ALMA9

- Seamless migration with minimal disruption, maintaining high availability and operational stability.

Enhanced Automation & Kubernetes Integration

- OSG Factory transition to Kubernetes enables automatic scaling, streamlined updates, and improved resilience across infrastructure components.

Efficiency Gains with Overloading Strategy

- Pilot overloading has improved CPU utilization, demonstrating better resource efficiency across various sites.

Future Plans

- Focused on further infrastructure modernization: token-based authentication, monitoring upgrades, HL-LHC readiness.

Ongoing Commitment to Stability

- Continuous collaboration with partners and adaptation to ensure robust CMS operations and scalability.

Spare slides

General CMS SI Overview

